

Depth Camera Distance Measurement

Depth Camera Distance Measurement

1. Course Content
2. Program Startup
3. Source Code Analysis

1. Course Content

- Run example programs, within the depth camera measurement range, use the depth camera to measure distances.

2. Program Startup

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

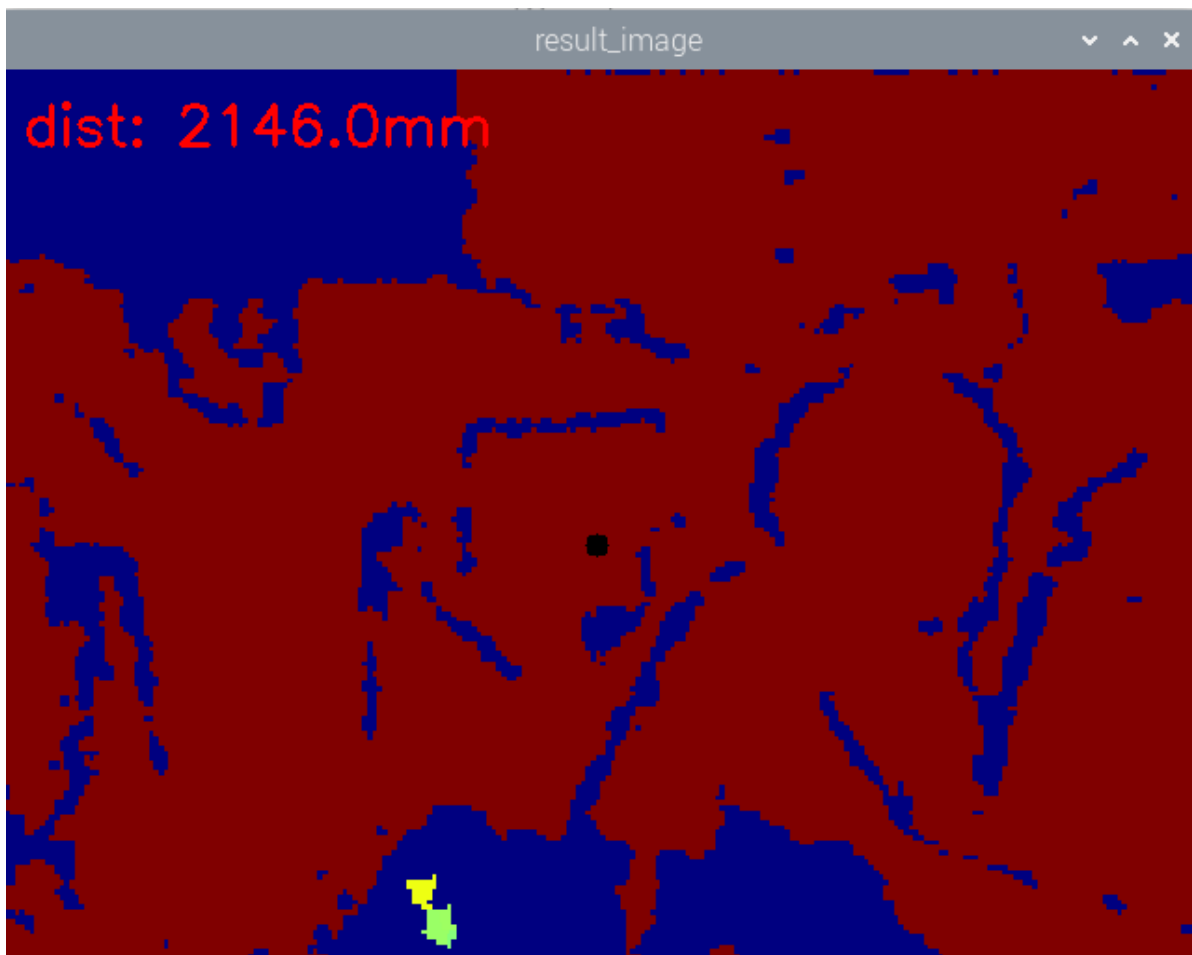
Open a terminal inside the container,

```
exec_m3
```

- Enter command in terminal:

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=Measure_Distance
```

After startup, as shown in the figure below, click on the red area with the mouse to select the measurement point. The selected point will turn black, and the distance measured by the depth camera will be directly printed in the upper left of the image, with units in millimeters.



3. Source Code Analysis

- Source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/Measure_Distance.py
```

- Import required library files

```
from cv_bridge import CvBridge
import cv2
from rclpy.node import Node
import rclpy
from sensor_msgs.msg import Image
import numpy as np
```

- Define depth image decoding format

```
encoding = ['16UC1', '32FC1']
```

- Define subscriber for depth image topic

```
self.sub_depth =
self.create_subscription(Image, "/camera/depth/image_raw", self.get_DepthImgCallBa
ck, 100)
```

- Define self.depth_bridge for converting message format to image format that OpenCV can process

```
self.depth_bridge = CvBridge()
```

- Convert image data message to image

```
depth_image = self.depth_bridge.imgmsg_to_cv2(depth_frame, encoding[1])
```

- Call OpenCV image processing function cv2.applyColorMap to convert depth map

```
depth_to_color_image = cv2.applyColorMap(cv2.convertScaleAbs(depth_image,
alpha=1.0), cv2.COLORMAP_JET)
```

- Convert image data Numpy array to 32-bit single-precision float array, this step ensures subsequent calculations use floating-point precision.

```
depth_image_info = depth_image.astype(np.float32)
```

- Get depth information of a specific point, in a 2D image, use (x,y) to determine the position of a point,

```
dist = depth_image_info[self.y,self.x]
```

- Process the obtained depth information and draw it on the image,

```
dist = round(dist,3)
dist = 'dist: ' + str(dist) + 'mm'
cv2.putText(depth_to_color_image, dist, (10, 40), cv2.FONT_HERSHEY_SIMPLEX,
1.0, (0, 0, 255), 2)
```

- Call OpenCV function to get mouse click events, click on points within the valid area of the image to get the xy values of that point,

```
cv2.setMouseCallback(self.window_name, self.click_callback)
def click_callback(self, event, x, y, flags, params):
    if event == 1:
        self.x = x
        self.y = y
```

- Draw the mouse-selected point on the image and display the image,

```
cv2.circle(depth_to_color_image,(self.x,self.y),1,(0,0,0),10)
cv2.imshow("result_image", depth_to_color_image)
```

